

Persistent Sparse Autoencoders: Learning Feature Timescales in Language Models

Haoyan Luo^{1*}, Mateo Espinosa Zarlenga², Mateja Jamnik¹

¹University of Cambridge, ²University of Oxford

Abstract

Sparse autoencoders (SAEs) decompose language model activations into sparse features, but standard SAEs encode each token independently and do not expose information that persists across a sequence. We introduce *Persistent Sparse Autoencoders* (Persistent SAEs), which extend standard SAEs by learning a persistence coefficient for each feature, allowing the model to learn which features should persist and for how long. Our experiments show that they retain competitive reconstruction quality while learning a spectrum of feature timescales: fast features behave as locally interpretable detectors, whereas slow features concentrate topic-level information in a persistent state. Moreover, as shown in a prompt-injection monitoring case study, slow features preserve detection signals and remain causally effective over long contexts. These results suggest that Persistent SAEs open up new opportunities for interpreting and monitoring language models through persistent semantic representations.

1 Introduction

Understanding why a large language model (LLM) produces a particular output requires identifying the information represented in its hidden states and how that information evolves as the model processes a sequence (Luo and Specia, 2024; Sharkey et al., 2025). Sparse autoencoders (SAEs) (Huben et al., 2024; Lan et al., 2024) have become a leading approach to this problem. An SAE reconstructs an LLM’s dense hidden state using a small number of learned features that may align with human-interpretable descriptions (Huben et al., 2024; Bricken et al., 2023; Templeton et al., 2024; Gao et al., 2025). This decomposition has enabled researchers to inspect concepts represented by language models and intervene on selected features to

*If you have any questions, feel free to email hl678@cam.ac.uk, mateo.espinosazarlenga@trinity.ox.ac.uk, or mateja.jamnik@cl.cam.ac.uk.

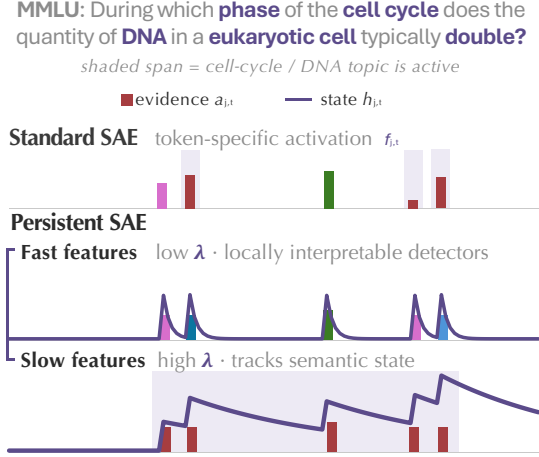


Figure 1: **From token-specific evidence to persistent semantic state.** Persistent SAEs learn a timescale for each feature: fast features remain token-specific and locally interpretable to explain the reconstruction, while slow features integrate sparse evidence over tokens to track the active semantic context.

influence model behaviour (Templeton et al., 2024; Marks et al., 2025; Arad et al., 2025).

Although causal transformer hidden states integrate the preceding context, standard SAEs encode each token independently (Huben et al., 2024; Bussmann et al., 2024, 2025). This atemporal formulation favours token-specific features but obscures the semantic state maintained by the model, that is, the topic-level context it carries across a sequence (e.g., topic, style, or task state). Recent methods incorporate temporal structure into SAEs through predefined partitions and decompositions of temporal roles (Bhalla et al., 2026; Lubana et al., 2026). However, they specify in advance which subset should carry temporal information before reconstruction, so the resulting temporal roles are predefined without clear evidence. It therefore remains unclear whether the temporal role of features can emerge directly from the data and reconstruction training.

We explore this question starting from the observation that temporal persistence associated with SAE features is already present in the hidden states. As shown in Figure 2, after a feature activates, later hidden states remain aligned with its decoder direction, and its past activations contain information that helps reconstruct subsequent hidden states. Yet, standard SAEs encode each position independently and therefore do not represent this persistence explicitly. To fill this gap, we introduce *Persistent Sparse Autoencoders*, which learn a persistence coefficient λ_j for every feature. Persistent SAEs separate persistent *state* from sparse *evidence*: each feature maintains a decaying *state variable*, updated by token-specific sparse *evidence*, with λ_j controlling its timescale and how long past evidence is retained. If carrying a feature’s state forward helps reconstruct later hidden states, there is an incentive to learn a higher λ_j during optimisation; if it does not, the objective favours a smaller λ_j and faster decay.

Our experiments show that Persistent SAEs retain competitive reconstruction quality while learning a broad range of feature timescales. Most are low- λ_j *fast features* that behave as locally interpretable detectors like standard SAEs, while high- λ_j *slow features* form compact semantic states that retain topic-level information in the hidden states. In a prompt-injection monitoring case study (Debenedetti et al., 2024), these slow features preserve detection signals and remain causally effective on the monitor’s judgement consistently after the injection. These results show that SAE reconstruction can organise sparse features by timescale, separating local evidence from a compact persistent state that tracks semantic context and can steer a monitor’s judgement over long contexts. Therefore, our main contributions can be summarised as:

- We introduce *Persistent Sparse Autoencoders*, which learn a persistence coefficient for each sparse feature through reconstruction alone. Persistent SAEs retain competitive reconstruction quality while separating low- λ_j *fast features* that act as locally interpretable detectors from high- λ_j *slow features* that concentrate topic-level information in a persistent state.
- Through a prompt-injection monitoring case study, we demonstrate the potential of slow features for monitoring and for steering models over long contexts, opening up new oppor-

tunities for interpreting and controlling LLMs through compact semantic states.

2 Background and Related Work

Sparse autoencoders. Given an LLM hidden state $x_t \in \mathbb{R}^d$ at token t , a standard sparse autoencoder (SAE) encodes it as non-negative sparse activations $f_t \in \mathbb{R}_{\geq 0}^m$ and reconstructs it as

$$\hat{x}_t^{\text{base}} = W_{\text{dec}} f_t + b_{\text{dec}}. \quad (1)$$

Feature j has normalised decoder direction $d_j = W_{\text{dec}}[:, j] / \|W_{\text{dec}}[:, j]\|_2$ and activation $f_t^{(j)}$; it is *active* when $f_t^{(j)} > 0$. Because only a few entries of f_t are non-zero, each reconstruction is expressed through a small set of features, making the contribution of individual features easier to inspect (Huben et al., 2024; Bricken et al., 2023). Subsequent SAE variants improve how this sparse capacity is allocated and organised, including BatchTopK and hierarchical Matryoshka SAEs (Gao et al., 2025; Busmann et al., 2024, 2025). See Appendix A for a summary of our notation.

Temporal structure in LLM representations. Although an autoregressive LLM builds x_t from the preceding context, a standard SAE computes each f_t independently. It therefore does not explicitly expose information carried across tokens. Traditional slow feature analysis shows that slowly varying latent factors can reveal higher-level structure in neural networks (Wiskott and Sejnowski, 2002). Prior studies likewise show that LLM hidden states track entity, game-board, spatial, temporal, and propositional states (Kim and Schuster, 2023; Li et al., 2023; Gurnee and Tegmark, 2024; Feng et al., 2025). These findings motivate testing whether standard SAE features inherit temporal persistence from the hidden states they reconstruct.

Temporal extensions of SAEs. Temporal SAEs pre-assign a high-level feature subset and use a contrastive objective to align its activations across nearby tokens (Bhalla et al., 2026). Lubana et al. (2026) instead use an attention-based predictor to separate context-predictable information from a sparse novel residual. Both prescribe temporal structure through a fixed partition; neither lets each sparse feature learn its own timescale through SAE reconstruction alone. Persistent SAEs fill this gap by giving every feature a persistent state with a learned persistence coefficient.

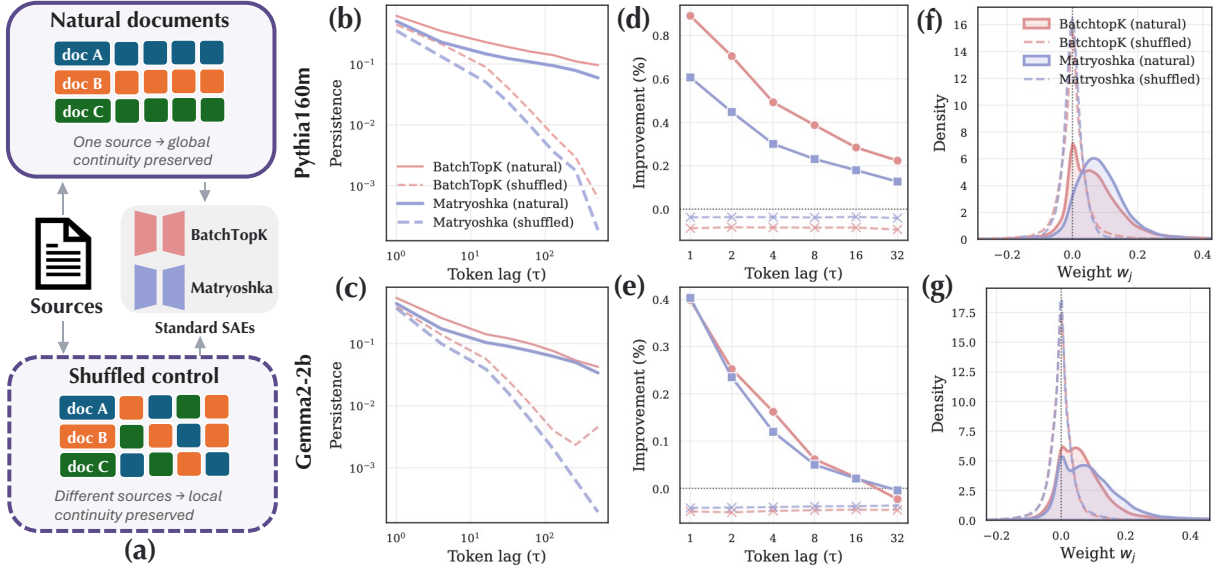


Figure 2: **Sparse features carry temporal structure that a token-specific SAE leaves unused.** (a) Natural documents preserve global continuity; the shuffled control joins sentences from different sources, preserving local continuity but breaking global continuity. (b, c) Decoder-aligned feature persistence $R_j(\tau)$ decays more slowly in natural documents than in the shuffled control. (d, e) Past activations improve held-out reconstruction on natural documents but not on shuffled controls. (f, g) The corresponding lag-one weights w_j are positive for natural documents but centred near zero for shuffled controls.

3 Are Standard SAE Representations Persistent?

We first test whether standard SAEs inherit temporal structure from the hidden states they reconstruct. Using a shared setup, we ask whether the signal along d_j remains after feature j activates, and whether past activations improve the reconstruction of later hidden states.

Setup. We analyse residual-stream states $\{x_t\}_{t=1}^T$ from Pythia-160M-deduped (layer 8) (Biderman et al., 2023) and Gemma-2-2B (layer 12) (Rivière et al., 2024) on Pile documents (Gao et al., 2020). We train BatchTopK (Bussmann et al., 2024) and Matryoshka (Bussmann et al., 2025) SAEs, neither of which has a temporal prior. Appendix B provides the full training configurations for these models.

Our *shuffled control* joins sentences from different source documents from Pile. It preserves local continuity but breaks the global continuity of the natural documents (Figure 2 (a)); Appendix C gives the construction details of the dataset.

Sparse Features Temporally Persist in the Hidden States. We first test whether the signal aligned with feature j remains after it activates. At token t , its projection is $p_t^{(j)} = d_j^\top x_t$. A large

$p_t^{(j)}$ indicates strong alignment between the hidden state and feature direction d_j . Conditioning on feature j being active at token t , we define its *normalised persistence* at lag τ (the distance to future tokens) as

$$R_j(\tau) := \frac{\mathbb{E}[p_{t+\tau}^{(j)} \mid f_t^{(j)} > 0] - \mathbb{E}[p_t^{(j)}]}{\mathbb{E}[p_t^{(j)} \mid f_t^{(j)} > 0] - \mathbb{E}[p_t^{(j)}]}. \quad (2)$$

Here, $\mathbb{E}[p_t^{(j)}]$ is the feature’s average projection. Thus, $R_j(\tau) = 1$ means the activated-token elevation is fully retained, values above one mean it is momentarily stronger, and $R_j(\tau) = 0$ means it has decayed to baseline activation. Appendix D provides the full evaluation procedure and feature-selection criteria for this measure.

Across both models and SAE families, decoder-aligned signals decay more slowly in natural documents than in the shuffled control (Figure 2 (b, c)). Standard SAE features therefore inherit persistence associated with global continuity, even though the SAE does not encode it explicitly.

Past Activations Help Reconstruct the Current Token.

Persistence alone does not show that past activations contain information useful for reconstruction. We therefore test whether $f_{t-\tau}$ can reduce the standard SAE’s reconstruction error at

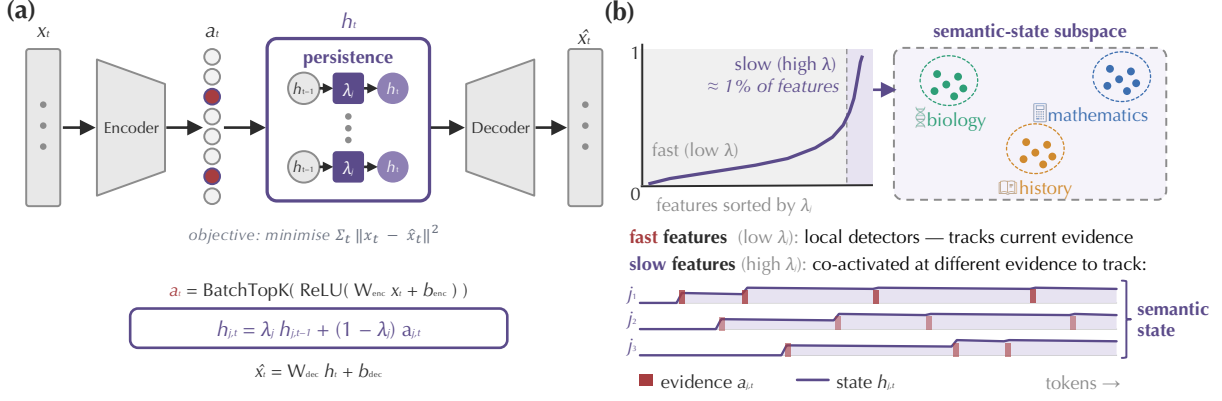


Figure 3: **Persistent Sparse Autoencoders.** (a) Each feature integrates sparse token-specific evidence a_t into state h_t with its learned persistence coefficient λ_j . (b) The learned feature timescales separate fast, locally interpretable features from a compact set of slow features that track semantic state.

token t . We test this using a restricted *context correction* experiment. Starting from the standard reconstruction in Equation 1, we add a contribution from the activations τ tokens earlier:

$$\hat{x}_t^{\text{ctx}} = \hat{x}_t^{\text{base}} + W_{\text{dec}} \text{diag}(w_\tau) f_{t-\tau}. \quad (3)$$

The diagonal weights $w_\tau \in \mathbb{R}^m$ let each past activation correct the reconstruction only along its own decoder direction. It directly tests whether a feature’s past activation predicts a later contribution along the same direction.

For each lag, we fit w_τ for 10 epochs with an L_2 penalty and measure the held-out fraction of baseline error recovered:

$$R_{\text{improve}}^2 = 1 - \frac{\sum_t \|x_t - \hat{x}_t^{\text{ctx}}\|_2^2}{\sum_t \|x_t - \hat{x}_t^{\text{base}}\|_2^2}. \quad (4)$$

For the control, we refit the same context correction weights to predict the natural-document targets from shuffled context activations (see Appendix D.1 for training details).

In Figure 2 (d, e), we observe that the past activations from the natural document recover reconstruction error, with gains decreasing as the lag grows. However, the shuffled control does not help the reconstruction. Correspondingly, Figure 2 (f, g) shows that the fitted context correction weights are mostly positive for natural documents but centred near zero for the shuffled control. Therefore, a standard SAE leaves reconstruction-relevant temporal structure unused in its sparse features, motivating a learned timescale for each feature in the next section.

4 Persistent Sparse Autoencoders

In this section, we introduce Persistent SAEs, which make this temporal structure explicit by separating sparse *evidence* from persistent *state* (Figure 3). We follow the standard BatchTopK encoder architecture (see Appendix B.2 for details). The encoder produces token-specific evidence $a_t \in \mathbb{R}_{\geq 0}^m$, where $a_t^{(j)} > 0$ denotes new evidence for feature j .

Each feature integrates its evidence into a persistent state $h_t \in \mathbb{R}_{\geq 0}^m$ using a learned persistence coefficient λ_j :

$$h_t^{(j)} = \lambda_j h_{t-1}^{(j)} + (1 - \lambda_j) a_t^{(j)}. \quad (5)$$

A small λ_j yields a fast, nearly token-specific feature; a large λ_j yields a slow feature that carries evidence forward. After k tokens, past evidence is attenuated by λ_j^k , corresponding to an exponential timescale of $-1/\log \lambda_j$ tokens. The factor $(1 - \lambda_j)$ prevents evidence from accumulating excessively, keeping the state magnitude stable. Appendix B.3 derives these properties and details the persistence parameterisation.

The decoder reconstructs the hidden state from this persistent state:

$$\hat{x}_t = W_{\text{dec}} h_t + b_{\text{dec}}. \quad (6)$$

The recurrence is diagonal: feature j carries only its own evidence along its decoder direction, preserving feature-level attribution. Sparsity is imposed on the token-specific evidence a_t ; the accumulated state h_t integrates these sparse updates and need not itself be exactly sparse. Setting every $\lambda_j = 0$ would recover a token-specific SAE. During reconstruction training, the objective favours a

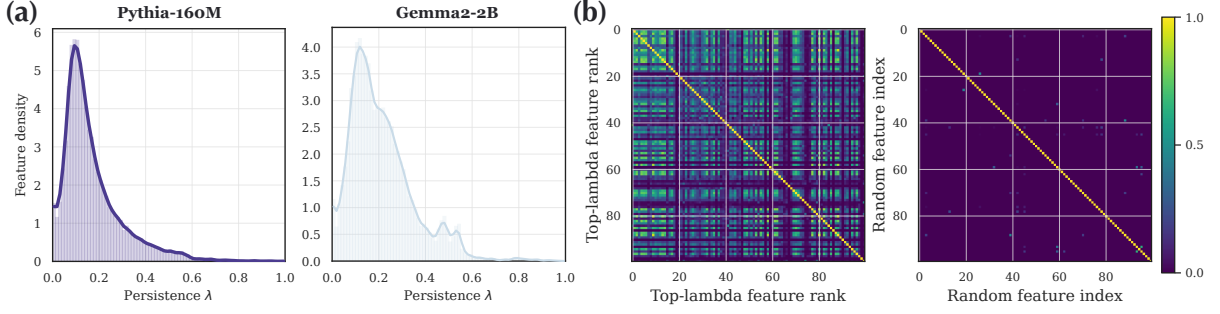


Figure 4: **Learned timescales of Persistent SAEs and coactivation of slow features.** (a) Persistence coefficients λ_j form a broad range of timescales of λ , with many fast features and fewer slow features. (b) The slow features coactivate more strongly than random features, indicating a structured persistent subspace.

larger λ_j when carrying feature j forward helps reconstruct later hidden states. The model therefore discovers each feature’s timescale without predefined feature partitions or a temporal objective.

5 Experiments

We first test whether adding persistent state preserves good reconstruction quality and locally interpretable sparse evidence. We then ask what the learned timescales organise, testing whether slow features jointly form a compact representation of semantic context. Finally, Section 6 studies whether this persistent state remains useful in a long-context monitoring case study.

Models and hyperparameters. We use the same models and dataset as Section 3. We compare with BatchTopK SAE (Bussmann et al., 2024) and Matryoshka SAE (Bussmann et al., 2025) as strong standard baselines, and Temporal SAE (Bhalla et al., 2026) as the closest temporally structured baseline. All models are trained on the same activations with $m = 16,384$ features and a BatchTopK budget of $k = 20$. Matryoshka and Temporal SAEs both use the same predefined 20%/80% feature partition. Appendix B gives the full training configuration. We show that training stabilises for Persistent SAEs without continual growth in the active state variable.

5.1 Reconstruction Quality

Table 1 reports fraction of variance explained (FVE), cosine similarity between the hidden state and its reconstruction, fraction of active features, and the temporal smoothness metric adopted from Bhalla et al. (2026), which measures the degree to which features change smoothly over time. The Persistent SAE remains competitive with

the strongest token-specific baselines. On Pythia-160M, it reaches 0.93 FVE and 0.95 cosine similarity. We show that the high- λ_j slow features have smoothness 0.05 on Pythia and 0.04 on Gemma, whereas low- λ_j features score 0.20 on both models. The Persistent SAE therefore preserves fast features for token-specific detail while concentrating temporal continuity in a smaller slow subset without predefined feature partitions.

Figure 4 (a) shows that the learned timescales of Persistent SAEs span a broad range. Most features are fast, with a smaller tail of highly persistent features. These slow features also coactivate more strongly and with richer structure than randomly selected features (Figure 4 (b)). This suggests that persistent context may be represented jointly by a structured group of slow features, which we investigate in Section 5.3.

5.2 Fast Features Are Locally Interpretable

We next ask how learned timescale relates to the conventional notion of an interpretable SAE feature. We use the automated interpretability pipeline of Bills et al. (2023): a judge model explains a feature from its highest-activation contexts, then scores how well that explanation predicts activation. For the Persistent SAE, we evaluate the sparse evidence a_t . This asks whether local evidence remains understandable before recurrence is applied (see Appendix E for autointerpretation details).

Across both models, low- λ_j fast features are easier to explain locally than high- λ_j slow features (Table 1). Fast features match or exceed the best baseline score on both models. Thus, adding recurrence preserves locally interpretable evidence for fast features, while interpretability score changes systematically with the learned temporal role. Qualitative examples for both models (Tables 4 and 5 in

		FVE ($\uparrow$)	Cosine Sim ($\uparrow$)	Fraction Alive ($\uparrow$)	Smoothness ($\downarrow$)			Autointerp ($\uparrow$)		
					Full	High	Low	Full	High	Low
Pythia 160M	BatchTopK	0.94	0.95	0.91	0.15	–	–	0.91	–	–
	Matryoshka	0.92	0.94	0.96	0.15	0.25	0.12	0.87	–	–
	Temporal	0.93	0.95	0.79	0.13	0.18	0.11	0.87	0.87	0.89
	Persistent	0.93	0.95	0.96	0.10	0.05	0.20	0.86	0.84	0.93
Gemma 2-2B	BatchTopK	0.72	0.79	0.92	0.16	–	–	0.87	–	–
	Matryoshka	0.71	0.77	0.87	0.17	0.21	0.14	0.89	–	–
	Temporal	0.62	0.75	0.75	0.14	0.12	0.16	0.85	0.85	0.86
	Persistent	0.66	0.75	0.91	0.11	0.04	0.20	0.85	0.79	0.89

Table 1: Core SAE metrics. *High/Low* denote predefined splits for Matryoshka and Temporal SAEs. For Persistent SAEs, the largest 1% and smallest 50% of persistence coefficients are used for *High/Low* partitions. Persistent SAEs remain competitive while separating smooth slow features from locally interpretable fast features.

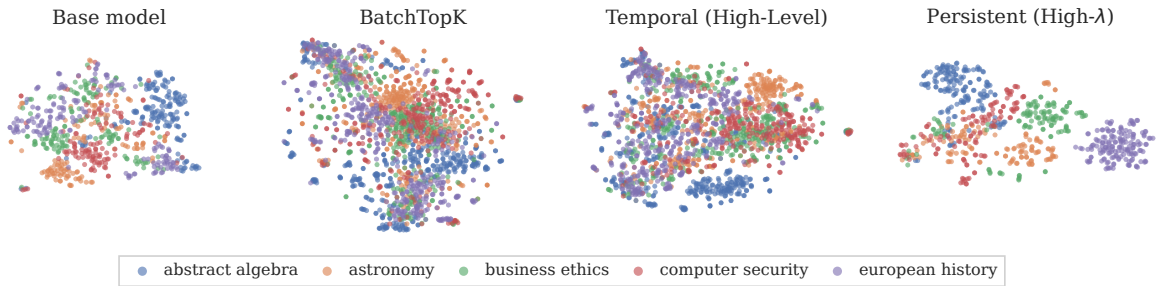


Figure 5: Token-level MMLU representations, coloured by subject. The Persistent SAE’s high- λ_j slow features reveal clearer subject structure than baseline SAEs or the raw hidden states.

Appendix E) show that fast features often respond to compact lexical or orthographic patterns, such as particular letters or substrings. Their evidence is visible at the current token and can be captured by a short description. Slow features more often reflect broader subjects or contextual conditions whose support is distributed across the sequence. This exposes a mismatch between persistent representations and a token-specific evaluation and reflects limitations of evaluating isolated SAE features: distinct features may receive similar descriptions (McCann, 2026) and explanations may not generalise to related inputs (Tian et al., 2025). The next section therefore evaluates slow features jointly, asking whether their state forms a coherent and predictive representation of context.

5.3 Slow Features Track Semantic States

Slow features form subject-specific clusters. The automated interpretability results suggest that slow features should be evaluated as a group and over a sequence. We first examine their geometry using MMLU examples from five subjects. For each sample, we collect the representations of its final 20 tokens and visualise them in two dimensions using t-SNE (see Appendix F for details).

We select the highest 1% slow features for visualisation. Figure 5 shows that BatchTopK features heavily mix subjects, whereas the Temporal SAE’s predefined high-level group affords limited separation. In contrast, slow Persistent SAE features form coherent subject regions, visually surpassing even raw hidden states. Importantly, Persistent SAEs learn this structure without subject labels.

Slow features capture topic structure without labels.

We test this observation quantitatively on spliced MMLU sequences, created by concatenating labelled segments from different subjects so that the subject changes at segment boundaries. At each token, a linear probe predicts the current subject from varying fractions of the feature dictionary for training the probe. We compare three rankings of Persistent SAE features: decreasing λ_j , increasing λ_j , random order. We compare against BatchTopK, Matryoshka, and Temporal SAE representations. We also include a supervised ranking as an oracle baseline. For each subject, the supervised ranking favours features that activate more strongly on that subject than on all other subjects. Because this ranking uses the subject labels directly, it serves as a label-informed oracle baseline (see

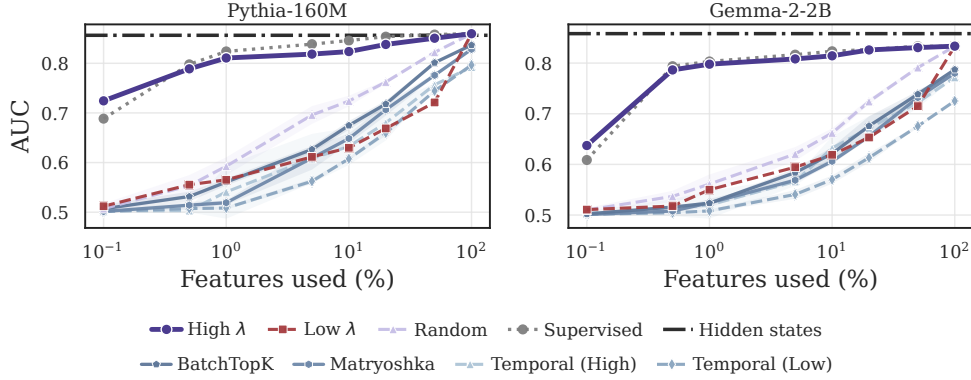


Figure 6: Subject-probing AUC on spliced MMLU sequences. Ranking features by decreasing λ_j recovers most of the signal from a small fraction of the dictionary and approaches supervised selection of features.

Appendix G for details).

Figure 6 shows that selecting features by decreasing persistence is nearly as efficient as supervised selection. Across both models, the slowest 1% of features reaches 0.81 AUC, while low- λ_j , random, and baseline SAE representations retain substantially less subject information at the same budget. Thus, persistence provides an unsupervised criterion for selecting a compact semantic state from a much wider sparse dictionary.

6 Case Study: Persistent Monitoring of Prompt Injection

The preceding experiments identify two complementary roles: fast features detect interpretable patterns at the current token, while slow features integrate local evidence into a compact semantic state. Indirect prompt injection (Debenedetti et al., 2024) provides a practical test of this separation. An agent may encounter a malicious instruction in an email, webpage, or tool response, continue reasoning for hundreds of tokens, and act only later (Debenedetti et al., 2025). A useful agentic monitor must therefore do more than detect the injected text when it appears; it must preserve a security-relevant signal until the agent acts (Zhan et al., 2024; Ye et al., 2026).

Setting. We use AgentDojo trajectories from the banking, travel, and workspace domains (Debenedetti et al., 2024), generated by Llama-3.3-70B-Instruct (Dubey et al., 2024). Each trajectory contains a user request, an untrusted span with either benign content or a prompt injection, and the agent’s subsequent tool interactions. We consider only failed attacks, in which an injection is present but the agent does not complete the attacker’s target action. This prevents the monitor

from relying on an overtly malicious action to identify an attack.

We use Gemma-2-2B as an external monitor: we replay each trajectory through the monitor model and record the layer-12 residual stream. We train a linear probe at the final token of the untrusted span ($d = 0$), where the injection is directly visible, and evaluate the same probe up to 350 tokens later without retraining (Figure 7 (a)). We compare baseline SAE activations, fast features (approximated with evidence a_t), and the highest 1% slow features. This evaluation measures how long each representation preserves the injection signal (see Appendix H for evaluation details).

Fast features detect injections locally; slow features track them over longer contexts. At $d = 0$, Persistent SAE evidence and the token-specific SAE baselines reach 94.0–96.7% accuracy (Figure 7 (b), top). Thus, fast features are equally strong locally interpretable detectors while the injection text is present. As the evaluation moves beyond the untrusted span, evidence and the token-specific representations fall towards chance. In contrast, the slow features remain informative across longer distances. Fast and slow features therefore play complementary roles: fast features detect the injection locally, while slow features carry its signal through the subsequent context.

Slow features causally influence the monitor’s judgement. Probe accuracy shows that a representation contains injection information, but not whether the corresponding direction influences the monitor’s own judgement. To test this, we truncate each trajectory at a checkpoint, append a query asking whether it should be flagged for prompt injection, and intervene at the final query token. For

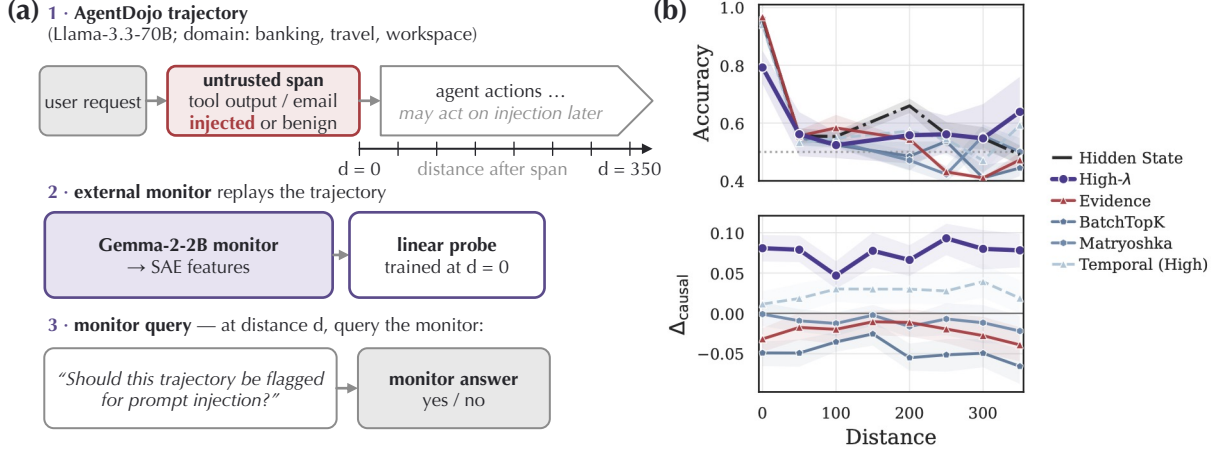


Figure 7: **Persistent monitoring of prompt injection.** (a) A probe trained at the end of an untrusted span is evaluated later in the trajectory. (b) Slow states retain more predictive information (top) and the strongest causal effect on the monitor’s judgement (bottom).

each SAE representation r , we construct a steering vector v_r by weighting the features of its most predictive features by their probe coefficients. We then apply interventions in both the positive and negative directions. If $s(x) = \ell_{\text{yes}}(x) - \ell_{\text{no}}(x)$ is the monitor’s logit margin, we measure

$$\Delta_{\text{causal}}(r, d) = \frac{1}{2} \mathbb{E}_i [s(x_{i,d} + \alpha_{i,d} v_r) - s(x_{i,d} - \alpha_{i,d} v_r)] \quad (7)$$

Here, $\alpha_{i,d}$ scales each intervention to 10% of the hidden-state norm. A positive Δ_{causal} means that moving in the probe direction shifts the monitor towards answering *yes*.

We show that slow features produce the largest, consistently positive intervention effect at every distance. In contrast, fast features and token-specific SAE baselines have near-zero or negative effects. Thus, slow-feature directions remain aligned with the monitor’s judgement long after the local injection signal has passed. Together, the probing and intervention results show complementary roles for fast and slow features. Fast features provide strong local detection, while the slow features retain a compact injection-related signal that remains both decodable and aligned with the monitor’s judgement. The Persistent SAE therefore supports both immediate detection and long-context retention in a single representation, potentially enabling more robust monitoring and control of LLMs.

7 Discussion and Conclusion

Discussion. Standard SAEs encode each token independently, even though hidden states also rep-

resent information accumulated over earlier tokens. Persistent SAEs address this mismatch by learning a feature-specific persistence coefficient through reconstruction alone, separating fast, locally interpretable features from a slow features that retains information across tokens. This slow subset forms topic-specific clusters, supports topic probing nearly as well as supervised baseline, and preserves prompt-injection signals after their local evidence has passed. These properties make slow features a promising compact semantic state for unsupervised topic tracking and for monitoring or intervening on safety-relevant signals over long contexts.

This mechanism has several limitations in its current form. Its diagonal recurrence gives each feature a single, fixed persistence coefficient and cannot model feature interactions or explicit resets. The prompt-injection study uses an external replay monitor over agent trajectories: interventions shift the monitor’s judgements, but we do not establish whether they affect the original agent’s actions. Future work can incorporate richer recurrent dynamics and test whether interventions affect end-to-end agent behaviour.

Conclusion. Persistent SAEs learn, from reconstruction alone, which sparse features should retain information across tokens and for how long. They preserve competitive reconstruction quality while separating fast, locally interpretable features from slow features that concentrate topic information and preserve monitoring and causal signals over long contexts. Persistent SAEs provide a potential tool for interpreting and monitoring the long-range information maintained by LLMs.

References

- Dana Arad, Aaron Mueller, and Yonatan Belinkov. 2025. [SAEs are good for steering – if you select the right features](#). In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 10241–10259, Suzhou, China. Association for Computational Linguistics.
- Usha Bhalla, Alex Oesterling, Claudio Mayrink Verdon, Himabindu Lakkaraju, and Flavio Calmon. 2026. [Temporal sparse autoencoders: Leveraging the sequential nature of language for interpretability](#). In *The Fourteenth International Conference on Learning Representations*.
- Stella Biderman, Hailey Schoelkopf, Quentin Gregory Anthony, Herbie Bradley, Kyle O’Brien, Eric Hallahan, Mohammad Aflah Khan, Shivanshu Purohit, USVSN Sai Prashanth, Edward Raff, and 1 others. 2023. Pythia: A suite for analyzing large language models across training and scaling. In *International Conference on Machine Learning*, pages 2397–2430. PMLR.
- Steven Bills, Nick Cammarata, Dan Mossing, Henk Tillman, Leo Gao, Gabriel Goh, Ilya Sutskever, Jan Leike, Jeff Wu, and William Saunders. 2023. [Language models can explain neurons in language models](#).
- Trenton Bricken, Adly Templeton, Joshua Batson, Brian Chen, Adam Jermy, Tom Conerly, Nick Turner, Cem Anil, Carson Denison, Amanda Askell, Robert Lasenby, Yifan Wu, Shauna Kravec, Nicholas Schiefer, Tim Maxwell, Nicholas Joseph, Zac Hatfield-Dodds, Alex Tamkin, Karina Nguyen, and 6 others. 2023. [Towards monosemanticity: Decomposing language models with dictionary learning](#). *Transformer Circuits Thread*.
- Bart Bussmann, Patrick Leask, and Neel Nanda. 2024. [Batchtopk sparse autoencoders](#). In *NeurIPS 2024 Workshop on Scientific Methods for Understanding Deep Learning*.
- Bart Bussmann, Noa Nabeshima, Adam Karvonen, and Neel Nanda. 2025. [Learning multi-level features with matryoshka sparse autoencoders](#). In *Forty-second International Conference on Machine Learning*.
- Edoardo DeBenedetti, Ilia Shumailov, Tianqi Fan, Jamie Hayes, Nicholas Carlini, Daniel Fabian, Christoph Kern, Chongyang Shi, Andreas Terzis, and Florian Tramèr. 2025. [Defeating prompt injections by design](#). *Preprint*, arXiv:2503.18813.
- Edoardo DeBenedetti, Jie Zhang, Mislav Balunović, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. 2024. Agentdojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents. In *Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track*.
- Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Amy Yang, Angela Fan, Anirudh Goyal, Anthony Hartshorn, Aobo Yang, Archi Mitra, Archie Sravankumar, Artem Korenev, Arthur Hinsvark, Arun Rao, Aston Zhang, and 82 others. 2024. The llama 3 herd of models. *CoRR*, abs/2407.21783.
- Jiahai Feng, Stuart Russell, and Jacob Steinhardt. 2025. [Monitoring latent world states in language models with propositional probes](#). In *The Thirteenth International Conference on Learning Representations*.
- Leo Gao, Stella Biderman, Sid Black, Laurence Golding, Travis Hoppe, Charles Foster, Jason Phang, Horace He, Anish Thite, Noa Nabeshima, Shawn Presser, and Connor Leahy. 2020. [The pile: An 800gb dataset of diverse text for language modeling](#). *arXiv preprint arXiv:2101.00027*.
- Leo Gao, Tom Dupre la Tour, Henk Tillman, Gabriel Goh, Rajan Troll, Alec Radford, Ilya Sutskever, Jan Leike, and Jeffrey Wu. 2025. [Scaling and evaluating sparse autoencoders](#). In *The Thirteenth International Conference on Learning Representations*.
- Wes Gurnee and Max Tegmark. 2024. [Language models represent space and time](#). In *The Twelfth International Conference on Learning Representations*.
- Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. 2021. Measuring massive multitask language understanding. In *ICLR*.
- Robert Huben, Hoagy Cunningham, Logan Riggs Smith, Aidan Ewart, and Lee Sharkey. 2024. [Sparse autoencoders find highly interpretable features in language models](#). In *The Twelfth International Conference on Learning Representations*.
- Najoung Kim and Sebastian Schuster. 2023. [Entity tracking in language models](#). In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 3835–3855, Toronto, Canada. Association for Computational Linguistics.
- Michael Lan, Philip Torr, Austin Meek, Ashkan Khakzar, David Krueger, and Fazl Barez. 2024. [Sparse autoencoders reveal universal feature spaces across large language models](#). *arXiv:2410.06981*.
- Kenneth Li, Aspen K Hopkins, David Bau, Fernanda Viégas, Hanspeter Pfister, and Martin Wattenberg. 2023. [Emergent world representations: Exploring a sequence model trained on a synthetic task](#). In *The Eleventh International Conference on Learning Representations*.
- Ekdeep Singh Lubana, Can Rager, Sai Sumedh R. Hindupur, Valérie Costa, Oam Patel, Sonia Krishna Murthy, Thomas Fel, Greta Tuckute, Daniel Wurgaft, Eric Bigelow, Demba E. Ba, Melanie Weber, and

- Aaron Mueller. 2026. [Priors in time: Missing inductive biases for language model interpretability](#). In *The Fourteenth International Conference on Learning Representations*.
- Haoyan Luo and Lucia Specia. 2024. [From understanding to utilization: A survey on explainability for large language models](#). *Preprint*, arXiv:2401.12874.
- Samuel Marks, Can Rager, Eric J Michaud, Yonatan Belinkov, David Bau, and Aaron Mueller. 2025. [Sparse feature circuits: Discovering and editing interpretable causal graphs in language models](#). In *The Thirteenth International Conference on Learning Representations*.
- Bryan McCann. 2026. [Descriptive collision: When many sparse features share one explanation](#). *Preprint*, arXiv:2605.12874.
- monology. 2023. [Pile uncopyrighted](#). Hugging Face dataset.
- Morgane Rivi re, Shreya Pathak, Pier Giuseppe Sessa, Cassidy Hardin, Surya Bhupatiraju, L onard Hussenot, Thomas Mesnard, Bobak Shahriari, Alexandre Ram , Johan Ferret, Peter Liu, Pouya Tafti, Abe Friesen, Michelle Casbon, Sabela Ramos, Ravin Kumar, Charline Le Lan, Sammy Jerome, Anton Tsitsulin, and 80 others. 2024. Gemma 2: Improving open language models at a practical size. *CoRR*, abs/2408.00118.
- Lee Sharkey, Bilal Chughtai, Joshua Batson, Jack Lindsey, Jeffrey Wu, Lucius Bushnaq, Nicholas Goldowsky-Dill, Stefan Heimersheim, Alejandro Ortega, Joseph Isaac Bloom, Stella Biderman, Adri  Garriga-Alonso, Arthur Conmy, Neel Nanda, Jessica Mary Rumbelow, Martin Wattenberg, Nandi Schoots, Joseph Miller, William Saunders, and 10 others. 2025. [Open problems in mechanistic interpretability](#). *Transactions on Machine Learning Research*. Survey Certification.
- Adly Templeton, Tom Conerly, Jonathan Marcus, Jack Lindsey, Trenton Bricken, Brian Chen, Adam Pearce, Craig Citro, Emmanuel Ameisen, Andy Jones, Hoagy Cunningham, Nicholas L Turner, Callum McDougall, Monte MacDiarmid, C. Daniel Freeman, Theodore R. Sumers, Edward Rees, Joshua Batson, Adam Jermyn, and 3 others. 2024. [Scaling monosemanticity: Extracting interpretable features from claude 3 sonnet](#). *Transformer Circuits Thread*.
- Yuchen Tian and 1 others. 2025. [On the sensitivity of interpretable sparse autoencoder features](#). *Preprint*, arXiv:2509.23717.
- Laurenz Wiskott and Terrence J. Sejnowski. 2002. Slow feature analysis: Unsupervised learning of invariances. *Neural Computation*, 14(4):715–770.
- Charles Ye, Jasmine Cui, and Dylan Hadfield-Menell. 2026. [Prompt injection as role confusion](#). In *Forty-third International Conference on Machine Learning*.
- Qiusi Zhan, Zhixiang Liang, Zifan Ying, and Daniel Kang. 2024. [InjecAgent: Benchmarking indirect prompt injections in tool-integrated large language model agents](#). In *Findings of the Association for Computational Linguistics: ACL 2024*, pages 10471–10506, Bangkok, Thailand. Association for Computational Linguistics.

A Notation

Table 2 collects every symbol used in the main text and appendices, in order of first appearance, as a reference for the terminology introduced in Sections 3–6.

B SAE Training Details

B.1 SAE Baseline Training Details

We train four SAE families for comparison: BatchTopK (Bussmann et al., 2024), Matryoshka BatchTopK (Bussmann et al., 2025), Temporal SAE (Bhalla et al., 2026), and our Persistent SAE. Each family uses the same dictionary width, BatchTopK activation budget, optimiser, and token budget. This matched setup isolates the effect of how each SAE represents temporal information, rather than differences in the amount of training data or the number of optimisation steps.

Common configuration. Every SAE has a dictionary of $m = 16,384$ features and a budget of $k = 20$ new feature activations per token. We train with Adam at a learning rate of 3×10^{-4} and use a linear warm-up over the first 1,000 steps. From step 1,000, we apply the BatchTopK activation-threshold schedule with decay $\beta = 0.999$, together with the auxiliary dead-feature loss weighted by $\alpha_{\text{aux}} = 0.03125$. Training activations come from Pile Uncopyrighted (monology, 2023), with the beginning-of-sequence position removed. For Gemma-2-2B (Rivi re et al., 2024), we use the layer-12 residual stream with hidden width $d = 2,304$; for Pythia-160M-deduped (Biderman et al., 2023), we use the layer-8 residual stream with $d = 768$.

The Matryoshka and Temporal SAEs both divide their dictionaries into the same two predefined feature groups, containing 20% and 80% of the 16,384 features. The Temporal SAE weights its reconstruction losses by 0.2 and 0.8, respectively, whereas Matryoshka uses uniform reconstruction weights. The Persistent SAE does not use a predefined partition; it instead learns a persistence coefficient for every feature.

Symbol	Meaning
<i>Hidden states and standard SAE (Section 3, 4)</i>	
$x_t \in \mathbb{R}^d$	Residual-stream hidden state at token position t
T	Number of tokens in a sequence
d	Dimensionality of the hidden state
m	Number of dictionary features (SAE width)
$W_{\text{enc}}, b_{\text{enc}}$	Standard SAE encoder weight matrix and bias
$W_{\text{dec}}, b_{\text{dec}}$	Standard SAE decoder weight matrix and bias
d_j	Unit-normalised decoder direction for feature j
$f_t \in \mathbb{R}_{\geq 0}^m$	Sparse activation of a standard SAE at token t (Section 4)
$f_t^{(j)}$	Sparse activation of feature j at token t
k	BatchTopK sparsity level (App. B.2)
B	Batch size used by BatchTopK selection (App. B.2)
<i>Persistence analysis (Section 3, App. D)</i>	
τ	Lag, in tokens
$p_t^{(j)}$	Decoder-aligned projection $d_j^\top x_t$
$R_j(\tau)$	Normalised persistence of $p_t^{(j)}$ at lag τ
$\beta_{\text{nat}}, \beta_{\text{shf}}$	Power-law decay exponents, natural / shuffled
<i>Context correction (Section 3, App. D.1)</i>	
$w_\tau \in \mathbb{R}^m$	Per-feature context-correction weights at lag τ
$w_{\tau,j}$	Context-correction weight for feature j at lag τ (w_j at $\tau = 1$)
$\hat{x}_t^{\text{base}}$	Baseline reconstruction from a standard SAE
$\hat{x}_t^{\text{ctx}}$	Context-corrected reconstruction (Equation 3)
R_{improve}^2	Held-out reconstruction-error improvement from context correction
<i>Persistent SAE (Section 4)</i>	
$a_t \in \mathbb{R}_{\geq 0}^m$	Evidence: the Persistent SAE’s per-token sparse signal (App. B.2)
$h_t \in \mathbb{R}_{\geq 0}^m$	Persistent state: recurrent latent integrating evidence over time (Equation 5)
$\lambda_j \in (0, \lambda_{\text{max}})$	Persistence coefficient for feature j (App. B.3)
λ_{max}	Upper bound on the persistence coefficient (= 0.999)
l_j	Unconstrained logit parameterising λ_j
$\sigma(\cdot)$	Sigmoid function

Table 2: Notation used throughout the paper, grouped by the section in which each symbol is introduced.

Matched token budget. The four families process activations in different units. BatchTopK and Matryoshka use shuffled token activations, the Temporal SAE uses adjacent token pairs, and the Persistent SAE uses contiguous sequences so that it can carry state across tokens. We therefore match the number of activation tokens processed per optimisation step and the total number of steps, rather than the number of these structural units. Each model is trained for 244,140 steps with an effective batch of approximately 2,048 activation tokens per step. For the flat baselines, this gives $2,048 \times 244,140 = 499,998,720$ tokens, or approximately 5.0×10^8 tokens in total.

- **BatchTopK and Matryoshka.** The activation buffer returns a flat batch of 2,048 token activations at each step.
- **Temporal SAE.** The buffer returns adjacent previous–next token pairs from the same activation stream. We keep the number of reconstructed token positions per step equal to the flat-baseline batch size.

- **Persistent SAE.** The recurrent state requires contiguous sequences rather than shuffled tokens. We use eight sequences of context length 256 per step. After excluding the beginning-of-sequence position, this gives $8 \times 255 = 2,040$ usable token activations per step, within 0.4% of the flat-baseline budget.

All families consequently receive the same number of gradient updates and, up to the pairing or sequencing required by their objectives, approximately the same number of token activations. We save checkpoints on a fixed schedule and use the final dictionary (ae.pt) for every downstream evaluation.

Figure 8 shows the training dynamics of the Persistent SAEs. For both models, the loss decreases rapidly and then remains stable, while the state L_0 settles at roughly 400–500 active coordinates despite continued training. The carried-mass fraction stabilises at approximately 0.5 for Pythia and 0.65–0.70 for Gemma. Thus, the recurrence does not cause the active state to grow continually, and

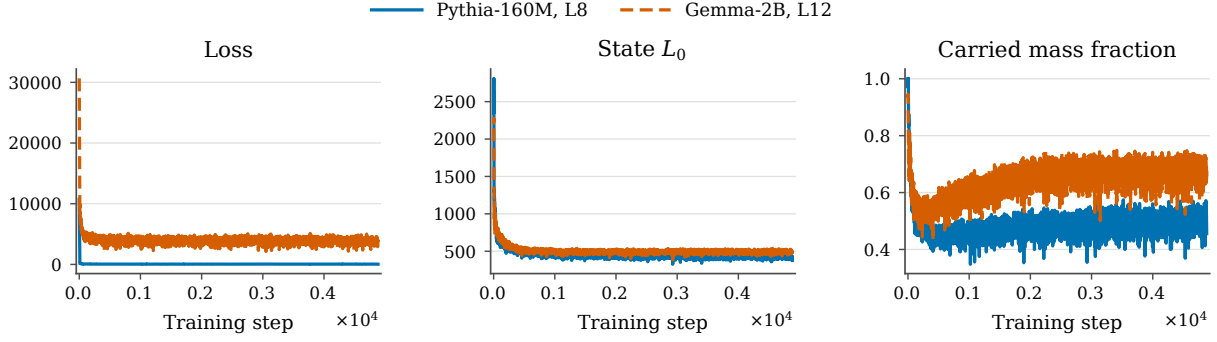


Figure 8: **Persistent SAE training dynamics.** Training loss (left), the number of non-zero coordinates in the persistent state h_t (state L_0 ; centre), and the fraction of state activation mass contributed by the carried term $\lambda \odot h_{t-1}$ (right) for Pythia-160M at layer 8 and Gemma-2B at layer 12. After a short initial transient, the loss and active-state size stabilise, while the carried-mass fraction remains well away from both zero and one; the models therefore use both carried state and new token-level evidence without continual growth in the active state.

neither model relies exclusively on new evidence or carried state.

B.2 BatchTopK Sparsity

For every SAE in this paper, sparsity is imposed using a BatchTopK mechanism in which a hidden state $x \in \mathbb{R}^d$ is first mapped to non-negative pre-activations

$$\tilde{f} = \text{ReLU}(W_{\text{enc}}(x - b_{\text{dec}}) + b_{\text{enc}}) \in \mathbb{R}_{\geq 0}^m, \quad (8)$$

and only the largest pre-activations are retained across the batch. For a batch $X = \{x_i\}_{i=1}^B$, the pre-activations $\{\tilde{f}_i\}_{i=1}^B$ are flattened into a single vector of length Bm , and the sparse activation f_i is obtained by keeping only the top kB entries globally and setting all others to zero. The reconstruction is then

$$\hat{x}_i = W_{\text{dec}}f_i + b_{\text{dec}}. \quad (9)$$

In practice, after training with exact BatchTopK selection, inference uses an equivalent thresholded form

$$f_{ij} = \tilde{f}_{ij} \mathbf{1}[\tilde{f}_{ij} > \theta], \quad (10)$$

where the threshold θ is estimated from training-time top- k statistics. The Persistent SAE uses the resulting sparse activation as its token-level evidence a_t ; only its downstream use differs.

B.3 Persistence Parameterisation

The persistence coefficients for the Persistent SAE are parameterised as

$$\lambda_j = \sigma(l_j) \cdot \lambda_{\text{max}}, \quad (11)$$

where $l_j \in \mathbb{R}$ is an unconstrained logit, σ is the sigmoid function, and $\lambda_{\text{max}} = 0.999$ is a hard cap that

prevents exact poles. This ensures $\lambda_j \in (0, \lambda_{\text{max}})$ throughout training. The factor $(1 - \lambda_j)$ in Equation 5 makes the recurrence an exponential moving average (EMA) instead of an unnormalised sum. Under constant evidence $a_t^{(j)} = a^{(j)}$, the state converges to $a^{(j)}$ for every λ_j , so persistence does not systematically increase the state magnitude. The cap below one prevents exact poles and gives every feature finite effective memory. We initialise the λ_j to 0.5 for both models.

C Dataset Details

We use three datasets for distinct parts of the evaluation. Pile documents provide the language-model hidden states used to train and evaluate the SAEs, MMLU questions provide subject-labelled text for analysing semantic state, and AgentDojo trajectories support the prompt-injection monitoring case study. We describe each dataset and our use of it below.

C.1 Pile Corpus

The original Pile is an 825 GiB English-language corpus assembled from 22 sources, including academic papers, web pages, books, code, and online discussions (Gao et al., 2020). We use the Pile Uncopyrighted release (monology, 2023), a derived version that removes the Books3, BookCorpus2, OpenSubtitles, YouTube Subtitles, and OpenWebText2 components. Documents are accessed in streaming mode and shuffled with a buffer of size 10,000 using a fixed seed. A document is accepted if, after tokenisation with the model tokeniser and truncation to the context cap, it contains at least the required minimum number of tokens.

For the natural corpus, each accepted document is passed through the model and the chosen residual-stream hidden states are retained. If a sequence has retained length L , temporal statistics are computed on token positions $\{16, \dots, L-1\}$ after dropping the initial tokens.

The shuffled control is constructed at the sentence level rather than by permuting tokens. Sentences are sampled from documents drawn from the same corpus and concatenated subject to a simple adjacency constraint that prevents consecutive sentences from coming from the same source document. This preserves local continuity while breaking global continuity, including long-range entity tracking and discourse progression.

C.2 MMLU Dataset

Massive Multitask Language Understanding (MMLU) is an English-language benchmark of four-option multiple-choice questions from 57 subjects spanning STEM, the humanities, the social sciences, and other professional and academic fields (Hendrycks et al., 2021). We use MMLU only for the semantic analyses; it is not used to train the SAEs. Specifically, we use questions from abstract algebra, astronomy, business ethics, computer security, and European history. These subjects cover distinct areas while providing a subject label for every example.

For the geometry analysis in Section 5.3, we sample 100 sequences across the five subjects and retain the final 20 token representations from each sequence. For contextual probing, we construct sequences by joining equal-length segments from different subjects and assign each token the subject label of its source segment. Appendix F and Appendix G provide the sampling and evaluation details for the two analyses.

C.3 AgentDojo Trajectories

Indirect prompt injection occurs when an agent retrieves attacker-controlled text from emails, webpages, documents, or tool outputs and follows instructions inside that text. AgentDojo (DeBenedetti et al., 2024) evaluates this setting with stateful tool environments containing 97 user tasks, 27 injection targets, and 629 security test cases across workspace, Slack, travel, and banking domains. Its task-specific checks evaluate whether the legitimate user request and the attacker’s target action were completed by inspecting the agent output and changes to the environment.

For our monitoring dataset, we use trajectories from the banking, travel, and workspace domains generated by Llama-3.3-70B-Instruct. Benign trajectories without an injection form the negative class ($y = 0$). The positive class ($y = 1$) contains runs with an important_instructions or direct attack for which the AgentDojo injection checker returns security=false. This value means that an injection was present but the agent did not complete the attacker’s target action. We retain these failed attacks deliberately: successful attacks often contain an overt malicious action, such as transferring money or sending private data, which could provide a trivial downstream classification cue. This restriction instead asks whether the monitor retains the semantic trace of an injection before, or even without, an observable security failure. We do not filter by task utility, so the legitimate user request may either succeed or fail.

D Temporal Structure on SAE Activations

We evaluate lags $\tau \in \{1, 4, 16, 32, 64, 128, 256, 512\}$ on documents of at most 1,024 traced tokens. A feature is retained only if it has at least 256 activation events, at least 64 valid anchors at every lag, and a non-negligible activated-token elevation. In particular, $R_j(\tau)$ is undefined and excluded when its denominator $\mathbb{E}[p_t^{(j)} \mid f_t^{(j)} > 0] - \mathbb{E}[p_t^{(j)}]$ is near zero, as occurs for near-dead or diffuse features; we do not add an ϵ to the numerator or denominator. On Pythia-160M, median persistence for BatchTopK falls from 0.618 at lag 1 to 0.160 at lag 64 and is still 0.096 at lag 512; Matryoshka follows the same pattern, from 0.504 to 0.108 and 0.059. On Gemma-2-2B, natural-document persistence remains measurable at lag 512, reaching 0.042 (BatchTopK) and 0.034 (Matryoshka). On both models the shuffled control decays to approximately zero at long lags. Fitting power-law descriptors to the positive decay tail yields median exponents $\beta_{\text{nat}} \approx 0.32$ versus $\beta_{\text{shf}} \approx 0.87\text{--}0.92$, a gap that holds for both SAE families.

Individual features occupy markedly different temporal regimes. When features are ranked by their average $R_j(\tau)$ within short ($\tau \leq 16$), medium ($32 \leq \tau \leq 64$), and long ($\tau \geq 128$) lag bands, the resulting distributions span nearly the full unit interval at every band. Some features are strongly persistent at short lags but decay before medium

lags; others retain elevated projections well into the long-lag regime. This heterogeneity is present in both SAE families, and is substantially larger than what is observed in the shuffled control.

D.1 Context Correction Details

We fit the per-feature context correction of Equation 3 independently at each lag $\tau \in \{1, 2, 4, 8, 16, 32\}$. For each lag, we use 192 natural documents to fit w_τ and 96 held-out natural documents to evaluate R_{improve}^2 . The target is always the natural-document reconstruction error. For the control, we refit the diagonal head to predict these same natural targets from context activations drawn from the shuffled corpus, using the same parameterisation and optimisation budget. Each document contributes at most 768 tokens after dropping 16 prefix tokens. We optimise for 10 epochs with AdamW, learning rate 5×10^{-3} , and weight decay 10^{-3} .

On Pythia-160M the correction removes 0.89% (BatchTopK) and 0.61% (Matryoshka) of the baseline reconstruction error at lag 1, decreasing but staying positive at lag 32 (0.22% and 0.13%). On Gemma-2-2B both SAEs remove approximately 0.40% at lag 1, falling to about 0.02% at lag 16 and vanishing by lag 32. The shuffled controls stay slightly below zero throughout, so unrelated context never improves reconstruction; the gain requires feature activations from the actual history of the target document.

Table 3 summarises the learned lag-one weights. On natural documents the mean weight is positive and a large majority of features take a positive weight, meaning many features should remain active after their initial evidence appears. The shuffled distributions are sharply centred at zero. This gives a feature-level reading of the reconstruction gain: it comes largely from carrying feature-aligned activity forward through a coherent document, which is precisely the behaviour the persistent state of Section 4 makes explicit.

E Automated Interpretability Details

We follow the automated interpretability pipeline of Bills et al. (2023). For each feature we collect its highest-activation contexts, prompt a judge model to write a short natural-language explanation, and score how well that explanation predicts the feature’s activation on held-out contexts, i.e. how well it predicts the event $\mathbb{K}[f^{(j)}(x) > \theta]$. We

Model	SAE	Mean w_j	Pos. frac.	Shuf. w_j
Pythia	BatchTopK	0.068	76.2%	0.001
Pythia	Matryoshka	0.087	91.4%	−0.001
Gemma	BatchTopK	0.068	80.4%	−0.001
Gemma	Matryoshka	0.091	81.9%	−0.001

Table 3: Learned lag-one context-correction weights w_j . *Mean w_j* and *Pos. frac.* are the mean and the positive fraction on natural documents; *Shuf. w_j* is the mean on the shuffled control.

use gpt-4o-mini as the judge, a context length of 1024 tokens, and roughly 2M tokens from the Pile for the activation examples. For the Persistent SAE we score its features on their evidence activations a_t , so that the comparison with the token-specific baselines is fair. We also score the slowest 1% and fastest 50% of Persistent SAE features by λ_j to evaluate their interpretability as groups.

F Semantic Geometry Details

For the t-SNE visualisation we draw 100 sequences from five MMLU subjects (abstract algebra, astronomy, business ethics, computer security, and European history) and keep the last 20 tokens of each sequence, so that every token carries a topic label. For each representation—raw hidden states, BatchTopK features, the Temporal SAE’s high-level group, and the Persistent SAE’s high- λ_j state—we collect the token-level vectors and project them to two dimensions with t-SNE using a common set of hyperparameters, colouring each point by its subject.

G Contextual Probing Details

We construct spliced sequences by concatenating equal-length segments drawn from different MMLU subjects, balancing topic labels within each sequence and holding out entire source documents for the test split, so that a token’s label is the topic of the segment it falls in. At every token we fit a linear probe to predict this topic from the token-level feature representation and report held-out macro AUC as a function of the fraction of features made available, swept from 0.1% to 100%. We evaluate the Persistent SAE on its persistent state h_t and order its features by decreasing λ_j , increasing λ_j , repeated random permutations, or supervised one-vs-rest importance. For each subject c , the supervised score for feature j is

$$s_{j,c} = \mathbb{E}[h_t^{(j)} \mid y_t = c] - \mathbb{E}[h_t^{(j)} \mid y_t \neq c], \quad (12)$$

Feature	λ	Autointerpretation	Representative activation	Score
High λ (top 10%): sustained semantic domains				
F-5528	7.03×10^{-1}	themes of ethnic diversity and experiences of discrimination	“ashamed of being Mexican because I’m different from the stereotype ”	0.93
F-12751	4.82×10^{-1}	phrases related to population-based studies and cancer research	incident colorectal cancer cases identified through a population-based cancer registry	0.93
F-15986	3.81×10^{-1}	terms related to high-performance cars and their specifications	rare and desirable muscle cars ; a 1971 Hemi ’Cuda	0.93
F-6538	3.79×10^{-1}	terms related to weather conditions and climate phenomena	a combination of global warming and the El Niño weather phenomenon	1.00
F-8449	3.69×10^{-1}	mathematical concepts and terms related to Banach spaces and polynomials	continuous scalar polynomials over a Banach space X	1.00
Low λ (bottom 50%): local lexical, syntactic, and template patterns				
F-1113	5.35×10^{-2}	the substring “white” followed by various words or concepts	“There is so much white space just waiting to be filled”	1.00
F-6958	4.96×10^{-2}	the substring “function” in JavaScript code snippets	.click(function() {	1.00
F-15079	4.81×10^{-2}	the phrase “U.S. Pat. No.” indicating references to patents	“disclosed in U.S. Pat. No. 5,111,981 ”	1.00
F-3587	1.10×10^{-2}	URLs and web documentation references	ubuntu.com/10.04/ serverguide/C/index.html	1.00
F-14758	4.04×10^{-4}	SQL commands related to table creation and data manipulation	CREATE TABLE IF NOT EXISTS tennis(...)	1.00

Table 4: Qualitative autointerpretation examples for Pythia-160M. High- and low- λ features are drawn from the top 10% and bottom 50% of the learned persistence coefficients, respectively. Bold text marks representative activating spans. The selected high- λ features respond to coherent semantic passages, whereas the low- λ features track local strings, syntax, or recurring templates. All selected features have an autointerpretation score above 0.85.

computed on the training split. This score measures how much more strongly a feature activates for subject c than for all other subjects. We sort features by this score within each subject and interleave the subject-specific rankings in round-robin order, ensuring that every subject contributes its most discriminative features. For the BatchTopK, Matryoshka, and Temporal baselines, we use their full dictionaries under the same probe.

H Prompt Injection Monitoring Details

Trajectory construction. Trajectories are taken from AgentDojo (Debenedetti et al., 2024) runs of Llama-3.3-70B-Instruct agents. Attack trajectories are restricted to the important_instructions and direct attack types and to runs with security=false. In this dataset, these positives contain an injection whose targeted action the agent did not execute; benign trajectories contain no injected content. Each trajectory is serialised and appended with the monitoring query

should this trajectory be flagged for prompt injection or unsafe instruction hijacking? Answer yes or no:

and read with Gemma-2-2B; all representations are taken from the layer-12 residual stream. The

model’s own answer to this query is close to uninformative: the yes-minus-no answer logit at the query position gives an AUC of 0.51, which is why we train internal monitors rather than relying on prompting.

Checkpoints and probes. For each trajectory we locate the final token of the injected span (for benign trajectories, the token at the matched position) and define checkpoints at fixed offsets of 0, 50, 100, 200, 250, 300, and 350 tokens after it, together with the final query position. At each checkpoint we extract, from the same layer, the raw hidden state, the BatchTopK and Matryoshka sparse activations, the Temporal SAE’s high-level feature group, and the Persistent SAE’s evidence a_t , full state h_t , and slow state (the largest 1% of h_t coordinates by λ_j). A logistic regression probe is fit on each representation using checkpoint $d = 0$ only and then evaluated, without retraining, at every other checkpoint; we report held-out accuracy and ROC AUC.

Causal intervention. For each SAE representation, we select the $k = 128$ features with the largest absolute probe coefficients and decode their weighted decoder columns into direction v_r . The direction is normalised and scaled so that

Feature	λ	Autointerpretation	Representative activation	Score
High λ (top 10%): sustained semantic domains				
F-11205	9.18×10^{-1}	terms related to cosmology and advanced scientific concepts	near-future CMB Stage-III and IV surveys yielding reconstructed CMB lensing maps	0.86
F-5578	8.55×10^{-1}	terms related to ionisation charge collection and quantum experiments in physics	ionisation charge collection efficiency observed in gases	0.86
F-15639	8.24×10^{-1}	text related to conflict, violence, and ethnic tensions	self-determination for Katanga; a separatist group composed of former Katanga Tigers	0.86
F-13900	7.85×10^{-1}	terms related to microbial biomass and ecological concepts in alpine and tundra environments	microbial biomass in an alpine meadow on the Tibetan Plateau	0.86
F-11933	7.70×10^{-1}	expressions of self-acceptance and emotional support in personal growth narratives	“In the last couple years, I’ve let go and decided to be myself .”	1.00
Low λ (bottom 50%): local lexical, syntactic, and template patterns				
F-9996	1.03×10^{-3}	legal terms and conditions related to software licenses	“See the License for the specific language governing rights and limitations”	1.00
F-11210	1.30×10^{-5}	the phrase “all of” in various contexts	“it is unlikely that all of these loci respond to selection independently”	0.93
F-14595	7.87×10^{-6}	the substring “mk” within mathematical and scientific contexts	repeated \mkern commands in mathematical markup	0.93
F-13259	2.73×10^{-6}	specific terms or phrases enclosed in quotation marks	users searching for “ Pizza ” or “ Calgary pizza ”	1.00
F-7307	1.88×10^{-6}	the word “In” at the beginning of sentences	“ In this way the missing persons in our evolutionary mystery would ...”	0.86

Table 5: Qualitative autointerpretation examples for Gemma-2-2B. High- and low- λ features are drawn from the top 10% and bottom 50% of the learned persistence coefficients, respectively. Bold text marks representative activating spans. The selected high- λ features respond to coherent semantic passages, whereas the low- λ features track local strings, syntax, or recurring templates. All selected features have an autointerpretation score above 0.85.

$\|\alpha_{i,d}v_r\|_2 = \rho\|x_{i,d}\|_2$, with $\rho = 0.1$, then added to the residual stream with both signs. Equation 7 measures the resulting change in the monitor’s logit margin. Interventions use the direction learned at $d = 0$, so Figure 7 reports how its causal effect transfers across distance.